

Ejercicio 13

Considere el siguiente fragmento de código:

```
int count = 0; int n = a.length;
for (int i = 1; i <= n; i = i*2) {
    for (int j = 0; j < n; j+=n/2) {
        a[j]++;
    }
}
```

Este algoritmo se ejecuta en una computadora que procesa 1.000 operaciones por segundo. Determine el tiempo **aproximado** (considerando el orden de ejecución del algoritmo) que requerirá el algoritmo para resolver un problema de tamaño $n=4096$. Datos que tenemos:

- En un segundo se realizan 1000 operaciones.
- Cantidad de operaciones de la que tenemos que calcular el tiempo que tarda en ejecutarse, $n = 4096$.

Primero hay que calcular el orden del fragmento de código.

En este algoritmo tenemos, en la primera línea, operaciones constante que vamos a representar con c_1 y dos for anidados que van a ser traducidas como una sumatoria con otra adentro.

- Analizamos el primer for
Se ejecuta desde 1 hasta n pero en cada iteración se multiplica por 2.
Cambio del índice

1. $i = 1$
2. $i = 1 \cdot 2 \cdot 2 = 2^2$
3. $i = 1 \cdot 2 \cdot 2 \cdot 2 = 2^3$
- $\vdots$

Paso general: k . $i = 2^{(k-1)}$ Como la sumatoria aumenta de uno en uno, hay que encontrar el límite superior de la sumatoria que se ajuste al último

valor que va a tomar el índice. Despejamos i .

$$\begin{aligned} n &= i \\ 2^{k-1} &= i \\ k-1 &= \log_2(i) \\ k &= 1 + \log_2(i) \end{aligned}$$

Entonces la primera sumatoria se ejecuta desde $i = 1$ hasta $1 + \log_2(i)$. La sumatoria queda:

$$\sum_{i=1}^{1+\log_2 n} (algo)$$

- Analizamos el segundo for
Se ejecuta desde $j = 0$ hasta el valor anterior más cercanos a n que pueda tomar j pero en cada iteración se le suma $\frac{n}{2}$ al índice.
Cambio del índice:

1. $j = 0$
2. $j = \frac{n}{2}$
3. $j = \frac{n}{2} + \frac{n}{2} = 2 \cdot \frac{n}{2} = n$

El segundo for se ejecuta 2 veces. La sumatoria queda

$$\sum_{j=0}^1 c_2$$

Entonces la función $T(n)$ queda:

$$T(n) = c_1 + \sum_{i=1}^{1+\log_2 n} \left(\sum_{j=0}^1 c_2 \right)$$

Ahora hay que eliminar todas las variables de cambio de índices. Propiedad a utilizar:

- $\sum_{i=0}^n c = (n - 1 + 1) \times c$

$$\begin{aligned} T(n) &= c_1 + \sum_{i=1}^{1+\log_2 n} \left(\sum_{j=0}^1 c_2 \right) \\ &= c_1 + \sum_{i=1}^{1+\log_2 n} (2 \cdot c_2) \\ &= c_1 + 2c_2 \cdot (\log_2(n) + 1 - 1) \\ &= c_1 + 2c_2 + 2c_2 \log_2 n \end{aligned}$$

El orden de la función de tiempo de ejecución para este algoritmo es de $\log_2 n$. Como es el tiempo **aproximado**, lo utilizamos para calcular la cantidad de operaciones para ese n .

$$\log_2 4096 = 12 \text{ operaciones}$$

Sabemos que se procesan 1000 operaciones en 1 segundo.

$$\begin{array}{lll} 1000 \text{ operaciones} & \text{_____} & 1 \text{ segundo} \\ 12 \text{ operaciones} & \text{_____} & x = 0.012 \text{ segundos} \end{array}$$

Por lo tanto, el tiempo aproximado para resolver ese problema es de 0.012 segundos.